TP Unidad 7 · NoSQL y nuevas formas de almacenamiento · Food Store

Bases de Datos II · Grupo 11 · Integrante: Ramiro Quiroga · Repositorio: `github.com/nachoceba/TP_Concurrencia_IA_Grupo11` (carpeta `Unidad 7/TP 8`) · 05/11/2026
Entorno de ejecución: MongoDB 8.0.4 + mongosh 2.14.1 · PostgreSQL 16.4 (PGlite) cargado con los scripts del repo `parcial_BD/01_schema.sql → 04_objects.sql → 02_data_demo.sql` (los totales los calculan los triggers reales).

1 · Subconjunto y modelo documental (Consigna 1)

Subconjunto «pedidos y detalle de pedidos». Tablas y claves que intervienen en PostgreSQL:

Tabla	PK	FK / relación	Qué aporta al documento
`pedido`	id_pedido	usuario_id → usuario	fecha, estado, forma_pago, total, metadatos, eliminado
`detalle_pedido`	id_detalle_pedido	pedido_id → pedido · producto_id → producto	arreglo `lineas`: cantidad, precio_unitario (congelado), subtotal
`usuario`	id_usuario	—	objeto `cliente` (sólo id, nombre, apellido)
`producto` / `categoria`	id_producto / id_categoria	producto.categoria_id → categoria	objeto `lineas[].producto` con su categoría

Colección `foodstore.pedido` — un documento por pedido:

```
{ id_pedido: 1,        // PK conservada (índice único)
  fecha: ISODate, estado: "TERMINADO",
  forma_pago: "EFECTIVO", metadatos: {...},
  total: Decimal128("13400.00"), eliminado: false,
  cliente: { id_usuario: 1, nombre, apellido }, // objeto
  lineas: [                // arreglo embebido
    { id_detalle_pedido: 1, cantidad: 2,
      precio_unitario: Decimal128("4500.00"),
      subtotal: Decimal128("9000.00"),
      producto: { id_producto: 1,
        nombre: "Muzzarella",
        categoria: { id_categoria: 1, nombre: "Pizzas" } } },
    ... ] }
```

Decisiones.

- **Identificadores conservados:** id_pedido, id_detalle_pedido, id_usuario, id_producto, id_categoria. Permiten cruzar cada documento con su fila de origen.
- **Importes en `Decimal128`**, equivalente a `NUMERIC(10,2)`: evita errores de redondeo de `double` al validar `cantidad × precio`.
- **Fecha como `ISODate`** (no texto), igual que el `TIMESTAMPTZ` de origen.
- Índice multikey en `lineas.producto.id_producto` para la consulta c.

2 · Migración (Consigna 2)

`foodstore.mongodb.js` crea la base `foodstore` y la colección `pedido` con su validador, y migra con `insertMany()` **los 7 pedidos y 16 líneas** del seed `02_data_demo.sql`. Todos los datos son reales del seed: no se agregaron pedidos de prueba. Cada documento incluye fecha, estado, cliente, productos, cantidades y precios históricos. El script es re-ejecutable (borra y recrea la colección). La consulta 0 de `consultas_postgresql.sql` arma desde PostgreSQL, con `jsonb_build_object`/`jsonb_agg`, el mismo documento que se insertó. Así se verifica que la migración es fiel a la base de origen (`evidencias/03`).

```
pedidos migrados: 7                      ← PostgreSQL: pedidos 7 · lineas 16
lineas migradas:  16
R1 OK, rechazado: Document failed validation
R2 OK, rechazado: Document failed validation        (evidencias/01_migracion_mongodb.txt)
```

3 · Reglas de integridad y qué se embebió (Consigna 3)

- **R1 · Subtotal coherente:** en cada línea `subtotal = cantidad × precio_unitario`, con el precio *histórico* de la venta y no el precio actual del producto. En PostgreSQL lo garantizan `trg_subtotal` y `ck_detalle_subtotal`. En MongoDB, un `$expr` con `$allElementsTrue` + `$map` en el validador.
- **R2 · Total coherente:** `total = Σ subtotales` de las líneas. En PostgreSQL lo garantiza `trg_total` / `calcular_total_pedido`. En MongoDB, `$expr: {$eq: ["$total", {$sum: "$lineas.subtotal"}]}`.

- Además: `id_pedido` único, `estado` y `forma_pago` limitados a los valores de los ENUM, `cantidad ≥ 1` entera, importes ≥ 0 y al menos una línea por pedido. Se probó insertar un subtotal y un total incoherentes, y ambos fueron **rechazados**.

Por qué embeber. Las **líneas** sólo existen dentro de su pedido, se leen siempre junto con él y no cambian después de la venta. Por eso se embeben, y así un pedido se lee y se escribe en una sola operación atómica. Del **producto** y del **cliente** se embebe una *copia mínima* (id + nombre) y no el registro completo. Es un dato histórico de la venta. El stock, la descripción o el mail cambian y siguen viviendo en sus tablas: si se embebieran habría que actualizar cada pedido. Por la misma razón se omitieron mail, celular y contraseña del cliente (no son esenciales para el pedido y son datos personales).

4 · Consultas en MongoDB (Consigna 4)

Consulta	Instrucción (`consultas.mongodb.js`)	Resultado (id_pedido)
a) Todos, con id y estado	`db.pedido.find({}, {_id:0, id_pedido:1, estado:1, fecha:1, total:1})`	7: 1, 2, 3, 4 (TERMINADO) · 5 (CONFIRMADO) · 6 (PENDIENTE) · 7 (CANCELADO)
b) Por estado	`find({estado: "TERMINADO", eliminado: false})`	4: 1, 2, 3, 4
c) Con un producto	`find({"lineas.producto.id_producto": 11, eliminado: false}, {"lineas.$": 1, ...})` (Coca-Cola Original 1.5L)	4: 1, 3, 4, 6
d) Total > umbral	`find({total: {$gt: NumberDecimal("14000")}})`	2: 2 ($29.000), 3 ($19.000)

Umbral de d). El ticket promedio de los pedidos migrados, calculado con `$avg` en la misma corrida, es **$14.214,29**. Se redondeó a $14.000 para obtener los pedidos que están por encima de lo habitual: los candidatos a un análisis de clientes de mayor gasto. El mismo promedio se obtiene en PostgreSQL con `avg(total)`. En c) se busca por `id_producto` y no por nombre: el ID es el identificador conservado y no depende de cómo esté escrito el nombre. La proyección `lineas.$` devuelve sólo la línea que coincidió.

```
== c) Pedidos con el producto 11 (Coca-Cola Original 1.5L)          (extracto de
evidencias/02_consultas_mongodb.txt)
[ { id_pedido: 1, estado: 'TERMINADO',
    lineas: [ { id_detalle_pedido: 2, producto: { id_producto: 11, nombre: 'Coca-Cola Original 1.5L', ... },
                cantidad: 2, precio_unitario: Decimal128('2200.00'), subtotal: Decimal128('4400.00') } ] },
  { id_pedido: 3, ... id_detalle_pedido: 8 ... }, { id_pedido: 4, ... id_detalle_pedido: 10 ... },
  { id_pedido: 6, estado: 'PENDIENTE', ... id_detalle_pedido: 15 ... } ]
cantidad c): 4
== Control: pedidos descuadrados (esperado 0)
0
```

5 · Verificación contra PostgreSQL (Consigna 5)

Consultas equivalentes en `consultas_postgresql.sql`: b) `WHERE estado = 'TERMINADO' AND eliminado = FALSE`; c) `JOIN detalle_pedido … WHERE d.producto_id = 11` con `count(DISTINCT id_pedido)` y `string_agg` de los IDs.

Consulta	Cantidad en PostgreSQL	Cantidad en MongoDB	IDs PostgreSQL / MongoDB	Coinciden los IDs
Estado seleccionado (TERMINADO)	4	4	1, 2, 3, 4 / 1, 2, 3, 4	Sí
Producto seleccionado (id 11)	4	4	1, 3, 4, 6 / 1, 3, 4, 6	Sí

También coinciden a) (7 pedidos con los mismos estados), d) (pedidos 2 y 3) y los 7 totales, que en PostgreSQL calculan los triggers (`evidencias/03`).

Diferencias detectadas y correcciones

La **primera versión del modelo** (commit `dec390d`) **no coincidía** con PostgreSQL. Se ejecutó para comparar (`evidencias/04`):

Problema en la versión 1	Efecto medido	Corrección
Datos escritos a mano («Hamburguesa Clásica», «Bebida Gaseosa»…), sin relación con el seed real	b) TERMINADO devolvía **[2]** contra **[1, 2, 3, 4]** en PostgreSQL. c) «Bebida Gaseosa» devolvía [4], pero ese producto no existe en Food Store (0 filas en PostgreSQL)	Migración desde `02_data_demo.sql` con los IDs, importes y estados de origen
Un único `producto` por pedido; `detalle_pedido` en otra colección sin `producto_id` ni id de línea	No se podían representar pedidos con varias líneas (6 de los 7 pedidos tienen 2 o 3) ni buscar productos «dentro del arreglo de líneas»	Arreglo `lineas` embebido con `producto` e `id_detalle_pedido`
Sin datos del cliente; fecha como texto	No cumplía la Consigna 2; los filtros por rango de fechas comparaban texto	Objeto `cliente` embebido; `ISODate`
Validador aplicado con `collMod` *después* de `insertMany`; sin regla de total	Los datos se cargaron sin validar. Una línea suelta en otra colección se aceptaba sin control de pedido ni total	`createCollection` con validador **antes** de insertar, con las reglas R1 y R2

Las consultas de PostgreSQL se limitan a `id_pedido ≤ 7` (el subconjunto migrado). Si en la base se ejecutó `06_transacciones.sql`, quedan pedidos de prueba extra (por ejemplo, el 8 sin líneas). Esos pedidos aparecerían sólo en PostgreSQL y no son un error de migración. Un pedido sin líneas, además, sería rechazado por el validador (`minItems: 1`).

6 · Evaluación con IA (Consignas 6 y 7)

Consulta realizada (Claude Opus 5.5, Anthropic, 05/11/2026; prompt completo y respuesta resumida en `ia_consulta.md`). El prompt describe el modelo relacional actual (pedido, detalle_pedido con precio congelado, triggers de subtotal y total), el **acceso frecuente al pedido completo** y la **necesidad de conservar la coherencia de cantidades y precios**, y pide comparar PostgreSQL y MongoDB. **Recomendación de la IA:** mantener PostgreSQL como sistema de registro y usar MongoDB, a lo sumo, como modelo de lectura.

Afirmación de la IA	Contraste (documentación oficial y laboratorio)	Veredicto
A1. «Embebiendo las líneas, el pedido completo se lee en una sola operación y su escritura es atómica: cabecera y líneas no pueden quedar a medio guardar.»	**Documentación:** la guía de *Embedding* indica que los datos embebidos permiten obtener los datos relacionados en una sola operación. El manual de atomicidad dice: «a write operation is atomic on the level of a single document, even if the operation modifies multiple embedded documents». **Laboratorio:** la consulta c devolvió pedido + línea + producto sin JOIN (en PostgreSQL hicieron falta 2 JOIN). Los inserts que rompían R1 y R2 se rechazaron enteros: el conteo siguió en 7 y no quedó ningún documento parcial.	Confirmada
A2. «MongoDB no tiene FKs ni triggers como PostgreSQL; la coherencia de cantidades, precios y totales queda a cargo de la aplicación.»	**Documentación:** el manual de *Schema Validation* admite validadores con `$jsonSchema` y con operadores de consulta, incluido `$expr`. **Laboratorio:** el validador rechazó `subtotal 8000 ≠ 2 × 4500` y `total 1.00 ≠ 9000` («Document failed validation»): la coherencia *dentro* del documento la impone la base, no la aplicación. Lo que sí es cierto: no hay FK (una línea podría citar un `id_producto` inexistente) y el validador sólo *verifica* el total, no lo *calcula* como `trg_total`.	Parcialmente correcta

Conclusión

Conviene **mantener PostgreSQL** como sistema de registro de los pedidos y no migrar este subconjunto. **Integridad transaccional:** confirmar un pedido toca `pedido`, `detalle_pedido` y el stock de `producto`. PostgreSQL lo resuelve con FK, CHECK, triggers y transacciones ACID en una sola instancia. En MongoDB el validador sólo protege el interior del documento (A2), y el resto exige transacciones multi-documento sobre un replica set. **Patrón de acceso:** el documento embebido responde «pedido completo» y «pedidos con un producto» sin JOIN (A1). Aun así, PostgreSQL devolvió exactamente los mismos IDs con índices sobre `detalle_pedido`, y hasta arma el mismo documento con `jsonb_build_object`. Para el volumen de Food Store, esa ganancia no compensa operar dos motores. **Costo de mantener dos copias:** duplicar los datos obliga a sincronizarlos. La primera versión del modelo ya mostró el riesgo: estados, productos y totales que no coincidían con la base real. Combinar tecnologías sólo se justificaría si la lectura del historial de pedidos superara ampliamente a la escritura. En ese caso, MongoDB sería una réplica de lectura alimentada desde PostgreSQL, nunca la fuente de verdad.

Archivos adjuntos

Archivo	Contenido
`foodstore.mongodb.js`	Creación de base y colección, validador (R1/R2), índices, `insertMany` de 7 pedidos y prueba de las reglas
`consultas.mongodb.js`	Consultas a–d (equivale a `consultas_mongodb.js`) y control de coherencia
`consultas_postgresql.sql`	Exportación del documento de origen, conteos y consultas equivalentes a–d
`evidencias/01-04` · `ia_consulta.md`	Salidas completas de cada ejecución · prompt y respuesta de la IA

Uso de IA en el trabajo: además de la consulta de la sección 6, se usó un asistente de IA para revisar el modelo y regenerar los scripts. Todo se aceptó sólo después de ejecutarlo en ambos motores. Los nombres de clientes son datos ficticios del seed de la materia.